

Pipeline Run Report

IDEA

Build a B2B mobile-first pharmacy ordering app where a retail pharmacist (the retailer) can browse medicines from multiple distributors, place orders, manage a cart, see active and closed orders, and view their store profile. After login, the retailer lands on a 4-tab dashboard: Home (offers carousel + medicine search + quick links to Distributors / Schemes / Generic Medicines / Scan Prescription / Outstanding Payments), Orders (Active / Closed sub-tabs), Cart (line items with distributor + price), Profile (name, license number, favourites, store location).

Decision: WROTE_FILES

TIMESTAMP	2026-05-02T18:53:46.760Z
TOTAL TIME	3 ms across 9 step(s)
VALIDATION	valid · 6 feature(s), 10 task(s)
SELECTED TASK	t-home-2 (developer, 4h) -> f-home-tab
QA	PASS
CYBERSECURITY	GO
FILES WRITTEN	4 file(s), 13,426 bytes

This report captures the full agent chain (CEO -> CTO -> Engineering Manager -> Developer -> QA -> Cybersecurity). Each agent's complete output is included in the per-agent sections.

Steps

#	Step	Status	Duration	Notes
1	ceo	ok	1 ms	
2	cto	ok	0 ms	
3	engineering-manager	ok	0 ms	
4	validation	ok	0 ms	6 features, 10 tasks
5	task-selection	ok	0 ms	picked t-home-2 (developer, 4h) under f-home-tab
6	developer	ok	1 ms	
7	qa	ok	0 ms	decision=PASS
8	cybersecurity	ok	0 ms	decision=GO
9	file-writer	ok	1 ms	4 file(s), 13426 bytes

Validation

Valid — 6 feature(s), 10 task(s).

Selected task

Task:

t-home-2 — Build the Home tab page at `app/dashboard/home/page.tsx`: greeting line with the retailer name, a search bar at the top that links to `/dashboard/search?q=...`, a horizontally scrollable offers carousel reading from `GET /api/offers`, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls `GET /api/profile` and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.

Assignee:

developer

Estimate:

4 h

Feature:

f-home-tab — Home tab

Files written

4 file(s), 13,426 bytes total.

Path	Bytes
<code>app/dashboard/home/page.tsx</code>	6,596
<code>app/dashboard/search/page.tsx</code>	2,146
<code>components/MedicineRow.tsx</code>	3,299
<code>components/SearchBar.tsx</code>	1,385

CEO

Mission: Let an independent retail pharmacist place orders to every medicine distributor they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they live in today.

OKRS

- Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile) that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within 14 days
- Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order in the first 30 days post-launch
- Surface an outstanding-payments tile on the Home tab so retailers can see distributor-wise balance without calling, reducing reconciliation calls per retailer by 70% in the same window

PRIORITIES

- Stand up the authenticated 4-tab dashboard shell (mock retailer login + tab navigation) so every later task has a place to land in the UI
- Build the Home tab: search bar at the top, offers carousel, and the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments)
- Build the Cart and Orders tabs end-to-end: cart line items with distributor + total, place-order action, and Active/Closed sub-tabs in Orders
- Build the Profile tab as a read-only retailer card (name, license number, favourites count, store location) plus the outstanding-payments stub it links to from Home

DELEGATION

CTO: Build a mobile-first Next.js 14 (App Router) + Tailwind webapp. Backend is Next.js Route Handlers in the same project; demo data lives in in-memory Maps seeded at boot (one pilot retailer, four distributors, ~20 medicines, three offers). Authentication is a mock cookie-based session: POST /api/auth/login takes a license number and sets a pharmacy-session cookie, middleware guards /dashboard/*. No real distributor integration, no real payments, no Postgres this cycle. Ship by end of day 8 with a runnable end-to-end loop: login -> search a medicine -> add to cart -> place order -> see it under Orders/Active -> view Profile.

CMO: Skip this cycle. Pilot is direct sign-up with the 6 retailers we already have a relationship with; no paid acquisition until cycle 2.

CFO: Back-of-envelope only: distributor commission percent x average order value x monthly orders per retailer. Re-validate before cycle 2; no spend this cycle other than the existing engineering team.

CPO: Own the product copy: empty states (no orders, empty cart, no offers), the 'outstanding payments' tile heading and per-distributor row, the 5 quick-link tile labels and supporting microcopy, and the order-status vocabulary (Placed / Acknowledged / Out for delivery / Delivered / Cancelled). Hand the copy sheet to CTO by end of day 2.

FULL CEO OUTPUT (JSON)

```
{
  "mission": "Let an independent retail pharmacist place orders to every medicine distributor they buy from in one mobile-first app, replacing the WhatsApp + phone-call ordering loop they live in today.",
  "okrs": [
    "Ship a retailer-only build with login and a 4-tab dashboard (Home, Orders, Cart, Profile) that supports the full search -> add-to-cart -> place-order -> see-in-active-orders loop within 14 days",
    "Convert 50% of the 6 pilot retailers from their current channel to at least one in-app order in the first 30 days post-launch",
  ]
}
```


ARCHITECTURE

Frontend:	Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout. Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on small screens that promotes to a side rail at md+. Client components only where state is needed (search input, cart updates, tab toggles); everything else is a server component reading from the in-process data layer.
Backend:	Next.js Route Handlers under app/api/* in the same Next project (no separate Node service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-end via shared zod schemas in lib/api/contracts.ts.
Database:	In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript types survive the swap; no migration this cycle.
Infrastructure:	Single Vercel deploy (frontend + Route Handlers behind one origin). Static assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for persistence, per-distributor HTTP integration via a simple fanout, and S3-compatible storage for prescription scans.

API CONTRACTS (11)

- POST /api/auth/login — Pilot login: takes a retailer license number, looks it up in the in-memory store, sets the pharmacy-session cookie (HttpOnly, SameSite=Lax) and returns the retailer summary. No password this cycle (single pilot retailer per device).
 - POST /api/auth/logout — Clears the pharmacy-session cookie.
 - GET /api/medicines/search — Case-insensitive substring search over medicine name, brand, and generic name. Empty q returns the full catalogue (used by the search results page when a retailer lands without a query). Each result carries the distributor it ships from so the UI can render the price + supplier chip.
 - GET /api/distributors — List all distributors the retailer can buy from. Used by the Home tab quick-link and the cart line items.
 - GET /api/offers — List of currently-active offers shown in the Home tab carousel. Returned in sortOrder. Each entry references the distributor running the offer.
 - GET /api/cart — Returns the current retailer's cart, grouped by distributor with a per-distributor subtotal and a grand total. Drives the Cart tab.
 - POST /api/cart/items — Add a medicine to the current retailer's cart, or increment the qty if it is already there. Validates that the medicineId exists. Returns the updated cart payload (same shape as GET /api/cart) so the Cart tab can render without a second round-trip.
 - DELETE /api/cart/items/{medicineId} — Remove a medicine from the current retailer's cart. Returns the updated cart payload.
 - POST /api/orders — Place orders from the current retailer's cart. The cart is fanned out into one order per distributor (so the retailer's UI can show each supplier's order separately under Orders > Active). Cart is cleared on success.
 - GET /api/orders — List the retailer's orders, filtered by status group. status=active returns placed | acknowledged | out_for_delivery; status=closed returns delivered | cancelled. Sorted newest-first.
 - GET /api/profile — Returns the current retailer's profile card plus the outstanding-payments breakdown shown on the Home tab tile.
-

DATABASE SCHEMA (8)

- **retailers**
 - id: string primary key (uuid in production)
 - name: string not null (display name shown on Profile)
 - owner_name: string not null
 - license_number: string not null unique (used as login id this cycle)
 - store_name: string not null
 - store_address: string not null
 - phone: string not null
 - email: string not null
 - gstin: string not null
 - favourites_medicine_ids: string[] not null default []
 - created_at: timestamp not null
 - **distributors**
 - id: string primary key
 - name: string not null
 - region: string not null
 - supplier_code: string not null unique
 - contact_phone: string not null
 - contact_email: string not null
 - rating: number not null (0-5, decimal)
 - created_at: timestamp not null
 - **medicines**
 - id: string primary key
 - name: string not null
 - brand: string not null
 - manufacturer: string not null
 - generic_name: string not null
 - distributor_id: string not null references distributors(id)
 - mrp_paise: integer not null (≥ 0)
 - selling_price_paise: integer not null (≥ 0)
 - scheme: string nullable (e.g. '10+1 free' or null)
 - pack_size: string not null (e.g. '10 tablets', '100 ml syrup')
 - hsn_code: string not null
 - created_at: timestamp not null
 - **offers**
 - id: string primary key
 - title: string not null
 - description: string not null
 - distributor_id: string not null references distributors(id)
 - banner_label: string not null (short text shown on the gradient banner)
 - valid_until: timestamp not null
 - sort_order: integer not null (lower = earlier in the carousel)
 - created_at: timestamp not null
 - **cart_items**
 - id: string primary key
 - retailer_id: string not null references retailers(id)
 - medicine_id: string not null references medicines(id)
 - distributor_id: string not null references distributors(id) (denormalised from medicines for easy grouping)
 - qty: integer not null (≥ 1)
 - unit_price_paise: integer not null (frozen at add-to-cart time)
 - added_at: timestamp not null
-

- unique(retailer_id, medicine_id)
- **orders**
 - id: string primary key
 - retailer_id: string not null references retailers(id)
 - distributor_id: string not null references distributors(id)
 - status: string not null (one of: placed | acknowledged | out_for_delivery | delivered | cancelled)
 - placed_at: timestamp not null
 - expected_delivery: timestamp nullable
 - total_paise: integer not null (≥ 0)
 - item_count: integer not null (≥ 1)
- **order_items**
 - id: string primary key
 - order_id: string not null references orders(id)
 - medicine_id: string not null references medicines(id)
 - qty: integer not null (≥ 1)
 - unit_price_paise: integer not null
 - line_total_paise: integer not null
- **outstanding_payments**
 - id: string primary key
 - retailer_id: string not null references retailers(id)
 - distributor_id: string not null references distributors(id)
 - amount_due_paise: integer not null (≥ 0)
 - last_updated_at: timestamp not null
 - unique(retailer_id, distributor_id)

RISKS (6)

- security: mock cookie auth (single shared session secret, no password, no multi-device invalidation) is intentional for the pilot but unsafe for production; flag for a real auth task in cycle 2
- compliance: pharmacy ordering is regulated (drug schedules, Schedule H1/X tracking, narcotics audit trail). This cycle ignores schedule classification and prescription validation; both are mandatory before any non-pilot rollout
- data loss: in-memory Maps reset on every process restart. Acceptable for a demo, blocking for a customer pilot
- scaling: medicine search is a linear scan over the in-memory store. Fine for ~20 medicines; needs Postgres trigram index or Meilisearch beyond ~1000 SKUs
- mobile UX: 'Scan Prescription' is a placeholder tile this cycle; barcode + OCR scan needs a native shell or WebRTC + a vision model and is out of scope
- outstanding-payments accuracy: the per-distributor balance is currently a static seed; integration with each distributor's ledger is a downstream task and the tile must be labelled as 'last reconciled at <ts>' to avoid retailer confusion

FULL CTO OUTPUT (JSON)

```
{
  "architecture": {
    "frontend": "Next.js 14 (App Router) + Tailwind CSS, mobile-first responsive layout.
Authenticated /dashboard route renders a fixed bottom tab bar (Home / Orders / Cart / Profile) on
small screens that promotes to a side rail at md+. Client components only where state is needed
(search input, cart updates, tab toggles); everything else is a server component reading from the
in-process data layer.",
    "backend": "Next.js Route Handlers under app/api/* in the same Next project (no separate Node
service). Auth uses a signed cookie set via Next's cookies() API; middleware at the project root
guards /dashboard/* and every /api/* route except /api/auth/login. All handlers are typed end-to-
end via shared zod schemas in lib/api/contracts.ts.",
    "database": "In-memory storage via lib/db/store.ts (Map<string, Record>), seeded at module
load with 1 pilot retailer, 4 distributors, ~20 medicines, 3 offers, and 2 outstanding-payment
rows. The schema is shaped to match a future Postgres + Drizzle migration so the same TypeScript
types survive the swap; no migration this cycle.",
    "infrastructure": "Single Vercel deploy (frontend + Route Handlers behind one origin). Static
assets via Vercel CDN. No external services this cycle. Future cycles: Neon Postgres for
```


Engineering Manager

FEATURES (6)

- f-shell-auth (high) — App shell + mock retailer auth
- f-data-spine (high) — In-memory data spine + shared types
- f-home-tab (high) — Home tab
- f-orders-tab (high) — Orders tab + place-order API
- f-cart-tab (high) — Cart tab + cart APIs
- f-profile-tab (medium) — Profile tab

TASKS (10)

- t-home-2 -> f-home-tab · developer · 4h - Build the Home tab page at `app/dashboard/home/page.tsx`: greeting line with the retailer name, a search bar at the top that links to `/dashboard/search?q=...`, a horizontally scrollable offers carousel reading from `GET /api/offers`, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and an outstanding-payments summary tile that calls `GET /api/profile` and shows the total + per-distributor breakdown. Mobile-first Tailwind layout, snap-scroll on the carousel.
- t-shell-1 -> f-shell-auth · developer · 4h - Set up the Next.js 14 + Tailwind project skeleton: `package.json` (next 14.2, react 18, tailwind 3, typescript 5, vitest), `tsconfig.json` with the `@/*` alias, `next.config.mjs`, `postcss.config.mjs`, `tailwind.config.ts` (mobile-first content globs), `app/globals.css` with the Tailwind directives + base background, `app/layout.tsx` with the `html/body` shell, and the public login page at `app/page.tsx` that posts to `/api/auth/login`. No auth handler yet (lands in `t-shell-2`); the page just calls `fetch` and redirects on success.
- t-shell-2 -> f-shell-auth · developer · 4h - Implement the mock auth: `POST /api/auth/login` (look up the retailer by license number against the in-memory store, set the pharmacy-session cookie `HttpOnly+SameSite=Lax` with a 30-day max-age), `POST /api/auth/logout` (clear the cookie), `middleware.ts` that gates `/dashboard/*` and every `/api/*` route except `/api/auth/login` (returns 401 JSON if missing cookie), a small `lib/auth/session.ts` helper that reads the current `retailerId` from the request cookies, and the authenticated `app/dashboard/layout.tsx` that renders the 4-tab bottom navigation (Home / Orders / Cart / Profile) and a stub `app/dashboard/page.tsx` that redirects to `/dashboard/home`.
- t-data-1 -> f-data-spine · developer · 4h - Implement the in-memory data spine: `lib/types.ts` (TypeScript types for Retailer, Distributor, Medicine, Offer, CartItem, Order, OrderItem, OutstandingPayment, OrderStatus, derived Money helpers) and `lib/db/store.ts` (Map-backed stores seeded at module import with 1 pilot retailer, 4 distributors, 20 medicines spanning the four distributors, 3 active offers, 2 outstanding-payment rows; CRUD helpers - `getRetailer`, `listDistributors`, `listOffers`, `searchMedicines(q)`, `getMedicine(id)`, `getCart(retailerId)`, `addCartItem`, `removeCartItem`, `clearCart`, `listOrders(retailerId, statusGroup)`, `createOrdersFromCart(retailerId)`, `getOutstandingForRetailer(retailerId)`).
- t-home-1 -> f-home-tab · developer · 3h - Implement the read APIs that drive the Home tab: `GET /api/medicines/search` (case-insensitive substring over name + brand + generic, returns each result with its distributor), `GET /api/distributors` (`id/name/region/rating` list), `GET /api/offers` (active offers in sort order, each with its distributor). All three use the auth-guarded session helper to identify the retailer (search results may carry per-retailer favourites later) and return 401 if the cookie is missing.
- t-cart-1 -> f-cart-tab · developer · 3h - Implement the cart APIs: `GET /api/cart` (returns the cart grouped by distributor with subtotals + grand total + itemCount, derived from the in-memory store), `POST /api/cart/items` (validates `medicineId` exists, increments qty if already present, returns the same grouped payload), `DELETE /api/cart/items/[medicineId]` (removes the line, returns the same grouped payload). All three resolve the retailer via the session helper and 401 without it.
- t-cart-2 -> f-cart-tab · developer · 3h - Build the Cart tab page at `app/dashboard/cart/page.tsx`: groups the cart by distributor (heading per supplier with subtotal), each line shows name + brand + qty stepper + remove button + line total, sticky footer with the grand total and a Place Order button

that POSTs to `/api/orders` and on success routes to `/dashboard/orders?status=active`. Empty-state copy when the cart is empty.

- t-orders-1 -> f-orders-tab · developer · 3h - Implement the orders APIs: GET `/api/orders?status=active|closed` (active = placed | acknowledged | out_for_delivery, closed = delivered | cancelled, sorted newest-first), POST `/api/orders` (reads the current retailer's cart, fans out into one order per distinct distributorId, copies cart_items into order_items, clears the cart, returns the new orderIds + count). 401 without the session cookie.
- t-orders-2 -> f-orders-tab · developer · 3h - Build the Orders tab page at `app/dashboard/orders/page.tsx` with Active and Closed sub-tabs (segmented control). Each sub-tab calls GET `/api/orders` with the matching status group and renders order cards (distributor name, status pill with status-color, item count, total, placed-at relative time, expected-delivery when present). Empty-state copy for both tabs.
- t-profile-1 -> f-profile-tab · developer · 3h - Implement GET `/api/profile` (returns the current retailer plus the outstanding-payments breakdown grouped by distributor) and build the Profile tab page at `app/dashboard/profile/page.tsx` that renders the retailer card (avatar initials, name, store name, license number, owner name, GSTIN, store address, phone, email, favourites count) and an outstanding-payments section listing each distributor's amount-due with the grand total. A logout button at the bottom POSTs `/api/auth/logout` and routes to `/`.

FULL ENGINEERING MANAGER OUTPUT (JSON)

```
{
  "features": [
    {
      "id": "f-shell-auth",
      "name": "App shell + mock retailer auth",
      "description": "Next.js 14 + Tailwind project skeleton, the login page, and the authenticated /dashboard layout with the bottom tab bar that every other feature lives inside.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-data-spine",
      "name": "In-memory data spine + shared types",
      "description": "Single source of truth for retailers, distributors, medicines, offers, cart items, orders, order items, and outstanding payments. In-memory Maps with seed data, plus the shared TypeScript types every Route Handler and page imports.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-home-tab",
      "name": "Home tab",
      "description": "Search bar at the top, offers carousel, the 5 quick-link tiles (Distributors, Schemes, Generic Medicines, Scan Prescription, Outstanding Payments), and the supporting catalogue / offers / distributors APIs.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-orders-tab",
      "name": "Orders tab + place-order API",
      "description": "Orders list page with Active and Closed sub-tabs, plus the POST /api/orders fanout (cart -> one order per distributor) and GET /api/orders status-grouped reader.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-cart-tab",
      "name": "Cart tab + cart APIs",
      "description": "Cart page grouped by distributor with per-distributor subtotal and a grand total, plus the GET / POST / DELETE cart endpoints that drive it.",
      "priority": "high",
      "status": "pending"
    },
    {
      "id": "f-profile-tab",
      "name": "Profile tab",
```


Developer

Task: t-home-2

IMPLEMENTATION PLAN

Land the Home tab and the search results destination it links to. `app/dashboard/home/page.tsx` is a server component that resolves the retailer via `getSession + getRetailer`, reads `listOffers + getOutstandingForRetailer` directly from the in-memory store (no extra HTTP hop since both run in-process), and renders 4 sections in mobile-first vertical stacking: greeting line with the retailer's first name + store name; client-side `<SearchBar />` with the magnifier icon; a horizontally snap-scrolling offers carousel with the gradient banner cards; a 2-column quick-links grid (Distributors / Schemes / Generic Medicines / Scan Prescription / Outstanding Payments) with inline SVG icons; an outstanding-payments amber tile that sums per-distributor balances and links to `/dashboard/profile` for the full breakdown. `components/SearchBar.tsx` is a small client component (controlled input + `onSubmit -> router.push to /dashboard/search?q=...`) used by both the home page and the search page. `app/dashboard/search/page.tsx` is a server component that reads `?q` from `searchParams`, calls `searchMedicines + getDistributor` directly, and renders a list of `<MedicineRow />` cards. `components/MedicineRow.tsx` is a client component (so the Add to cart button can call `/api/cart/items` and surface optimistic 'Added' feedback). All routes set `dynamic = 'force-dynamic'` because they read cookies via `getSession`.

FILES PRODUCED (4)

- `app/dashboard/home/page.tsx` (6,596 bytes)
- `app/dashboard/search/page.tsx` (2,146 bytes)
- `components/MedicineRow.tsx` (3,299 bytes)
- `components/SearchBar.tsx` (1,385 bytes)

TESTS (2)

- `components/SearchBar.tsx` renders the search input and routes to `/dashboard/search` on submit.
- `components/MedicineRow.tsx` renders the price, scheme badge, and Add to cart button.

NOTES (4)

- The home page reads `listOffers + getOutstandingForRetailer` directly from the in-memory store rather than going through `/api/offers / /api/profile`. This is the idiomatic Next 14 server-component pattern: same-process synchronous data access is faster than an internal HTTP round-trip and avoids the cookie forwarding problem (server fetch does not carry the user's cookie unless explicitly proxied).
- The 5 quick-link tiles use placeholder href targets (`/dashboard/distributors`, `/dashboard/schemes`, `/dashboard/generics`, `/dashboard/scan`) for the four non-shipping links + `/dashboard/profile` for outstanding payments. The four placeholder pages are deliberately out of scope for this task; cycle 2 fills them.
- `MedicineRow` is a client component because the Add to cart button needs `onClick + useTransition`. The home page itself stays a server component so it can read the session + store synchronously without serializing data into a client bundle.
- Outstanding-payments tile reads the data directly from the store (same justification as offers). The tile's 'View all' link routes to `/dashboard/profile`, where the per-distributor breakdown lives in the Profile tab from `t-profile-1`.

FULL DEVELOPER OUTPUT (JSON)

```
{
  "taskId": "t-home-2",
  "implementationPlan": "Land the Home tab and the search results destination it links to. app/dashboard/home/page.tsx is a server component that resolves the retailer via getSession + getRetailer, reads listOffers + getOutstandingForRetailer directly from the in-memory store (no extra HTTP hop since both run in-process), and renders 4 sections in mobile-first vertical stacking: greeting line with the retailer's first name + store name; client-side <SearchBar />"
}
```


QA

Decision: **PASS**
Test plan: 11 item(s)
Results: 11 pass, 0 fail
Bugs: 0

BUGS (0)

(none)

FULL QA OUTPUT (JSON)

```
{
  "taskId": "t-home-2",
  "testPlan": [
    "Verify the file set lands the Home tab + search destination + the two reusable client components: app/dashboard/home/page.tsx, app/dashboard/search/page.tsx, components/SearchBar.tsx, components/MedicineRow.tsx.",
    "Verify app/dashboard/home/page.tsx is a server component (no 'use client') that resolves the retailer via getSession + getRetailer, redirects to / on either miss, and reads listOffers + getOutstandingForRetailer directly from the in-memory store.",
    "Verify the home page renders the 4 documented sections: greeting (retailer first name + store name), search bar, offers carousel, quick-links grid, outstanding-payments tile.",
    "Verify the offers carousel uses snap-x snap-mandatory horizontal scrolling with min-w cards (mobile-first horizontal scroll).",
    "Verify the 5 quick-link tiles have the documented labels: Distributors, Schemes, Generic medicines, Scan prescription, Outstanding payments. The Outstanding payments tile links to / dashboard/profile.",
    "Verify the outstanding-payments amber tile sums per-distributor balances, formats with paiseToRupees, lists each distributor row, and shows an empty-state when there is no balance.",
    "Verify components/SearchBar.tsx is a client component, accepts an optional initialQuery prop, and on submit routes to /dashboard/search?q=<encoded>.",
    "Verify components/MedicineRow.tsx is a client component that POSTs to /api/cart/items on the Add to cart button and surfaces an inline error when the response is non-200, plus an optimistic 'Added' label that auto-resets.",
    "Verify the discount % is rendered only when MRP > selling price (no negative or zero discount displayed).",
    "Verify app/dashboard/search/page.tsx is a server component that reads ?q from searchParams, calls searchMedicines + getDistributor directly, and renders an empty-state when zero results.",
    "Search files[] content for forbidden placeholder strings."
  ],
  "results": [
    {
      "test": "File set covers the Home tab + search destination + reusable components.",
      "status": "pass",
      "details": "4 files: app/dashboard/home/page.tsx, app/dashboard/search/page.tsx, components/SearchBar.tsx, components/MedicineRow.tsx. Total ~7 KB of source."
    },
    {
      "test": "Home page is a server component with the documented session guard.",
      "status": "pass",
      "details": "No 'use client' directive on app/dashboard/home/page.tsx. getSession() returns Session | null; null -> redirect('/'). getRetailer(session.retailerId) returns Retailer | undefined; undefined -> redirect('/'). Defence in depth on top of the dashboard layout's own guard."
    },
    {
      "test": "All 4 sections present on the home page.",
      "status": "pass",
      "details": "Section 1 (greeting): 'Welcome back,' + first name + store name. Section 2 (search bar): <SearchBar /> render. Section 3 (offers carousel): aria-labelledby=offers-heading + ul.flex.snap-x.snap-mandatory + li with min-w-[280px]. Section 4 (quick links): aria-labelledby=quick-links-heading + grid grid-cols-2 + 5 entries. Section 5 (outstanding): aria-labelledby=outstanding-heading + amber tile + per-distributor list."
    },
    {
      "test": "Carousel uses snap-x snap-mandatory and min-width cards.",
      "status": "pass",
      "details": "ul classNames: 'flex snap-x snap-mandatory gap-3'. Each li has 'snap-start ... min-w-[280px] max-w-[280px]'. Outer wrapper has overflow-x-auto with -mx-4 px-4 to bleed off the page edge. Mobile-first behaviour confirmed."
    }
  ]
}
```


Cybersecurity

Decision:

GO

Summary:

Audited the Home tab UI and the search results destination it links to. The change ships two server pages (home + search) and two client components (SearchBar + MedicineRow). Server components read the session and the in-memory store synchronously; client components only call documented authenticated endpoints (POST /api/cart/items). No critical or high-severity vulnerabilities were found. One medium-severity item (search query reflected back to the page heading) is recorded; React's escaping makes it non-exploitable but flagging as a defence-in-depth note. Two low-severity items round out the audit.

Prompt-injection risk:

None. No file in the change set issues an LLM call, performs LLM-mediated I/O, renders untrusted text as raw HTML (React escapes all text interpolation), or executes shell or filesystem operations. The q query string flows only through server-side .toLowerCase() + .includes() against in-memory rows and into React text nodes which are escaped before reaching the DOM.

VULNERABILITIES (3)

MEDIUM

app/dashboard/search/page.tsx renders the user-supplied q query string directly into the page heading via JSX text interpolation: `Results for "\${q}"`. React escapes text-content interpolation by design (q never reaches the DOM as raw HTML), so this is non-exploitable today. The medium rating is a defence-in-depth flag: any future change that switches this template to dangerouslySetInnerHTML or to a markdown renderer would immediately become reflected XSS, because q is only trimmed and is not length-capped or sanitised on the server.

- Recommendation: Add a server-side max-length cap on q (e.g. q.slice(0, 80)) before rendering to defang both reflection-into-future-renderer and resource-exhaustion vectors. Pair with a /docs/ code-review checklist note that the q variable is untrusted and must never be passed to a raw-HTML sink.

LOW

components/MedicineRow.tsx posts to /api/cart/items and surfaces server.error in inline UI text without scrubbing. If a future server change ever passes an attacker-influenced string into the error field of the response (e.g. a database error message that includes user input), it would render as plain text on the page. Currently the only error sources are typed JSON validators with hardcoded messages; risk is theoretical.

- Recommendation: Cycle 2 hardening: enforce in lib/api/responses.ts that error fields are drawn from a closed enum of safe-to-display strings (e.g. 'invalid request', 'medicine not found', 'unauthorised'), and rely on logs for any operator-relevant detail. Also keep the existing fall-through string ('Add to cart failed (status)') so accidental HTML in error never reaches the DOM.

LOW

The home page server component reads the session and the store directly without any error boundary above the children. A bug in lib/db/store.ts that throws synchronously would crash the entire dashboard route to a Next 500 page, leaking the stack trace to anyone with the dashboard cookie. Not an externally-reachable surface today (cookie is required), but worth recording.

- Recommendation: Add an app/dashboard/error.tsx error boundary that renders a friendly retry UI. Cycle 2 hardening; non-blocking for the pilot.

REQUIRED FIXES (0)

(none)

FULL CYBERSECURITY OUTPUT (JSON)

```

{
  "taskId": "t-home-2",
  "summary": "Audited the Home tab UI and the search results destination it links to. The change ships two server pages (home + search) and two client components (SearchBar + MedicineRow). Server components read the session and the in-memory store synchronously; client components only call documented authenticated endpoints (POST /api/cart/items). No critical or high-severity vulnerabilities were found. One medium-severity item (search query reflected back to the page heading) is recorded; React's escaping makes it non-exploitable but flagging as a defence-in-depth note. Two low-severity items round out the audit.",
  "vulnerabilities": [
    {
      "severity": "medium",
      "description": "app/dashboard/search/page.tsx renders the user-supplied q query string directly into the page heading via JSX text interpolation: `Results for \"${q}\"`. React escapes text-content interpolation by design (q never reaches the DOM as raw HTML), so this is non-exploitable today. The medium rating is a defence-in-depth flag: any future change that switches this template to dangerouslySetInnerHTML or to a markdown renderer would immediately become reflected XSS, because q is only trimmed and is not length-capped or sanitised on the server.",
      "recommendation": "Add a server-side max-length cap on q (e.g. q.slice(0, 80)) before rendering to defang both reflection-into-future-renderer and resource-exhaustion vectors. Pair with a /docs/code-review checklist note that the q variable is untrusted and must never be passed to a raw-HTML sink."
    },
    {
      "severity": "low",
      "description": "components/MedicineRow.tsx posts to /api/cart/items and surfaces server.error in inline UI text without scrubbing. If a future server change ever passes an attacker-influenced string into the error field of the response (e.g. a database error message that includes user input), it would render as plain text on the page. Currently the only error sources are typed JSON validators with hardcoded messages; risk is theoretical.",
      "recommendation": "Cycle 2 hardening: enforce in lib/api/responses.ts that error fields are drawn from a closed enum of safe-to-display strings (e.g. 'invalid request', 'medicine not found', 'unauthorised'), and rely on logs for any operator-relevant detail. Also keep the existing fall-through string ('Add to cart failed (status)') so accidental HTML in error never reaches the DOM."
    },
    {
      "severity": "low",
      "description": "The home page server component reads the session and the store directly without any error boundary above the children. A bug in lib/db/store.ts that throws synchronously would crash the entire dashboard route to a Next 500 page, leaking the stack trace to anyone with the dashboard cookie. Not an externally-reachable surface today (cookie is required), but worth recording.",
      "recommendation": "Add an app/dashboard/error.tsx error boundary that renders a friendly retry UI. Cycle 2 hardening; non-blocking for the pilot."
    }
  ],
  "promptInjectionRisk": "None. No file in the change set issues an LLM call, performs LLM-mediated I/O, renders untrusted text as raw HTML (React escapes all text interpolation), or executes shell or filesystem operations. The q query string flows only through server-side .toLowerCase() + .includes() against in-memory rows and into React text nodes which are escaped before reaching the DOM.",
  "decision": "GO",
  "requiredFixes": []
}

```


